HOOX · PYNE · DOCUMENTATION

PLATFORM ENGINEERS · RELEASE OPERATORS

DEVOPS MANUAL

CI, Docker, Nuitka, metadata crypto, GCP, security

Open-source Pine Script evaluation framework

2026-10-25 DIN A4 · HTTPS://HOOX.SH/PYNE/DOCS OPEN SOURCE

CONTENTS

01 Overview

02 Local Dev

03 Ci

04 Release

05 Docker

06 Nuitka Build

07 Metadata Crypto

08 Observability

09 Security

OVERVIEW

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "DevOps" description: "CI matrices, containers, Nuitka LSP binaries, metadata crypto, and operational invariants for PYNE."

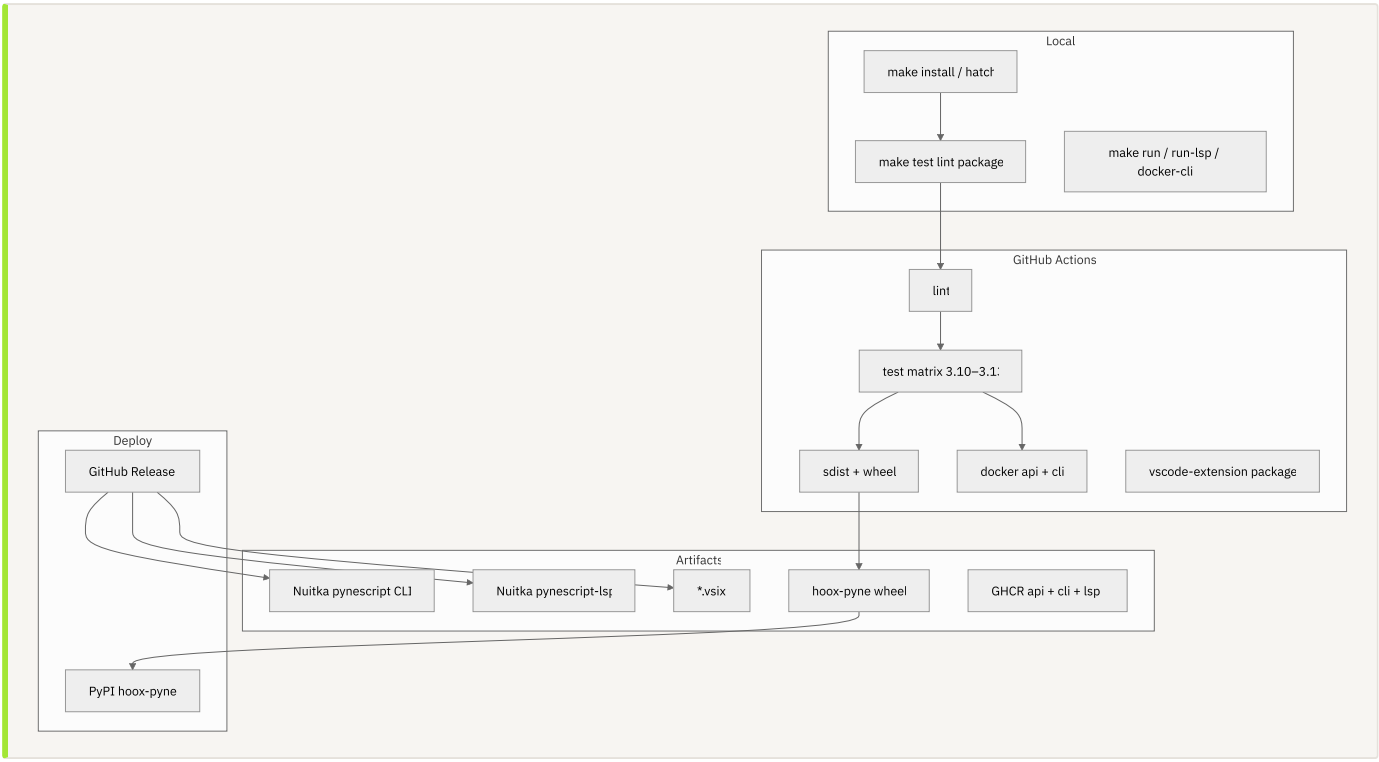
DEVOPS

Abstract

PYNE is a **multi-surface product**: pure-Python library + Click CLI (`pyne` / alias `pynescript`), pygls Language Server (`pyne-lsp`), Flask Pro API, VS Code extension (`hoox-sh.pyne` 0.4.5; the Python package is 0.6.22), and Docker targets (`api` / `cli` / `lsp`). Edge workers and the AXIS charting PWA are **sister repos** — not built in this tree.

This tab documents the **operational graph**: local loops, GitHub Actions, release tags, Docker images, Nuitka CLI/LSP binaries, Fernet-encrypted LSP metadata, observability hooks, and security controls.

Conceptual model



Invariant: CI green on `main` is necessary but not sufficient for a full release. Tag `v*` fires **Build & Release** (binaries + VSIX + GitHub Release), **Publish** (`hoox-pyne` from `__about__.py`, currently 0.6.22, to PyPI), and **GHCR** (`ghcr.io/hoox-sh/pyne/{api,cli,lsp}`).

Interface surface

Concern	Entry	Doc
Local install & loops	`Makefile`, hatch envs	Local development
PR / push CI	`.github/workflows/ci.yml` (`name: CI`)	CI
Versioned CLI + LSP + VSIX	`.github/workflows/release.yml` (`name: Build & Release`)	Release
PyPI (`hoox-pyne`)	`.github/workflows/publish.yml` (`name: Publish`)	Contributing
GHCR images	`.github/workflows/ghcr.yml` (`name: GHCR`)	Docker
Containers	`Dockerfile` targets api/api-dev/lsp/**cli**	Docker
Compiled CLI / LSP	`scripts/build/compile.py --target`	Nuitka build
Builtin metadata crypto	Fernet key + `.enc`	Metadata crypto
Logs / health	Flask `/`, gunicorn	Observability
Auth, CORS, secrets	`backend/middleware/auth.py`	Security

Internals (repo map)

Path	Role
`Makefile`	Human-facing orthography: install, test, lint, build, docker, worker
`pyproject.toml`	Hatch envs (`test` , `lint` , `docs`), optional extras (`lsp` , `data`)
`.github/workflows/`	`ci.yml` , `release.yml` , `publish.yml` , `ghcr.yml`
`scripts/build/`	Nuitka compile + CI build + Fernet metadata stage
`scripts/generate_builtin_metadata.py`	Regenerates LSP `builtin_metadata.json` from live builtins
`Dockerfile` / `docker-bake.hcl`	Multi-target Buildx images (api, api-dev, lsp)
`docker-compose.yml`	Local API (+ optional Redis / LSP profiles)
`vscode-extension/`	Node 22 extension package / vsce

Invariants & edge cases

1. **Two console scripts, two entrypoints.** Preferred: `pyne` (Click) ≠ `pyne-lsp` (pygls). Aliases: `pynescript` / `pynescript-lsp` . Ops scripts must not conflate them.
2. **Generated artifacts are not hand-edited.** ANTLR/ASDL under `generated/` and `builtin_metadata.json` are code-derived.
3. **Fernet key is gitignored.** Without a stable `CRYPTO_KEY` / `METADATA_KEY` secret, every CI encrypt produces a different `.enc` blob (harmless functionally, bad for reproducibility).
4. **Python matrix vs Nuitka pin.** CI tests 3.10–3.13; Nuitka release currently pins **3.11** (`release.yml` `PYTHON_VERSION` , `nuitka>=2.5.1,<2.8`).
5. **AXIS CI is not in this repo.** Playwright / PWA security gates live in hoox-sh/axis. There is no `axis-nightly.yml` here.

Worked examples

```
# Fast local confidence loop
make install
make lint
make test-lsp
make build-check   # import check only, ~30s, no Nuitka compile

# API in Docker — local *dev* stack (api-dev + source mounts)
make docker-up
make docker-smoke

# Production image bake (load local) + optional prod compose overlay
make docker-build
# export ADMIN_TOKEN=… && make docker-prod
```

Failure modes

Symptom	Likely cause	Fix
CI lint red, local green	Different ruff/mypy versions	Match CI install pins in workflow
Metadata decrypt fails in binary	Missing key at runtime	Set `PYNESCRIPT_METADATA_KEY` or embed key at build
Cloud Run 502 after deploy	Image missing deps / gunicorn bind	Confirm `Dockerfile` target `api` ENTRYPOINT + `PORT` / `GUNICORN_BIND`
VSIX empty / missing	`npm ci` / vsce not run	Use `make build-vscode` or release job artifacts
Nuitka Anaconda link error	Static libpython missing	`conda install libpython-static` or keep `--static-libpython=no`

See also

- Contributing
- LSP architecture
- Pro API contract
- pyne-worker
- pyne-agent-worker

LOCAL DEV

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

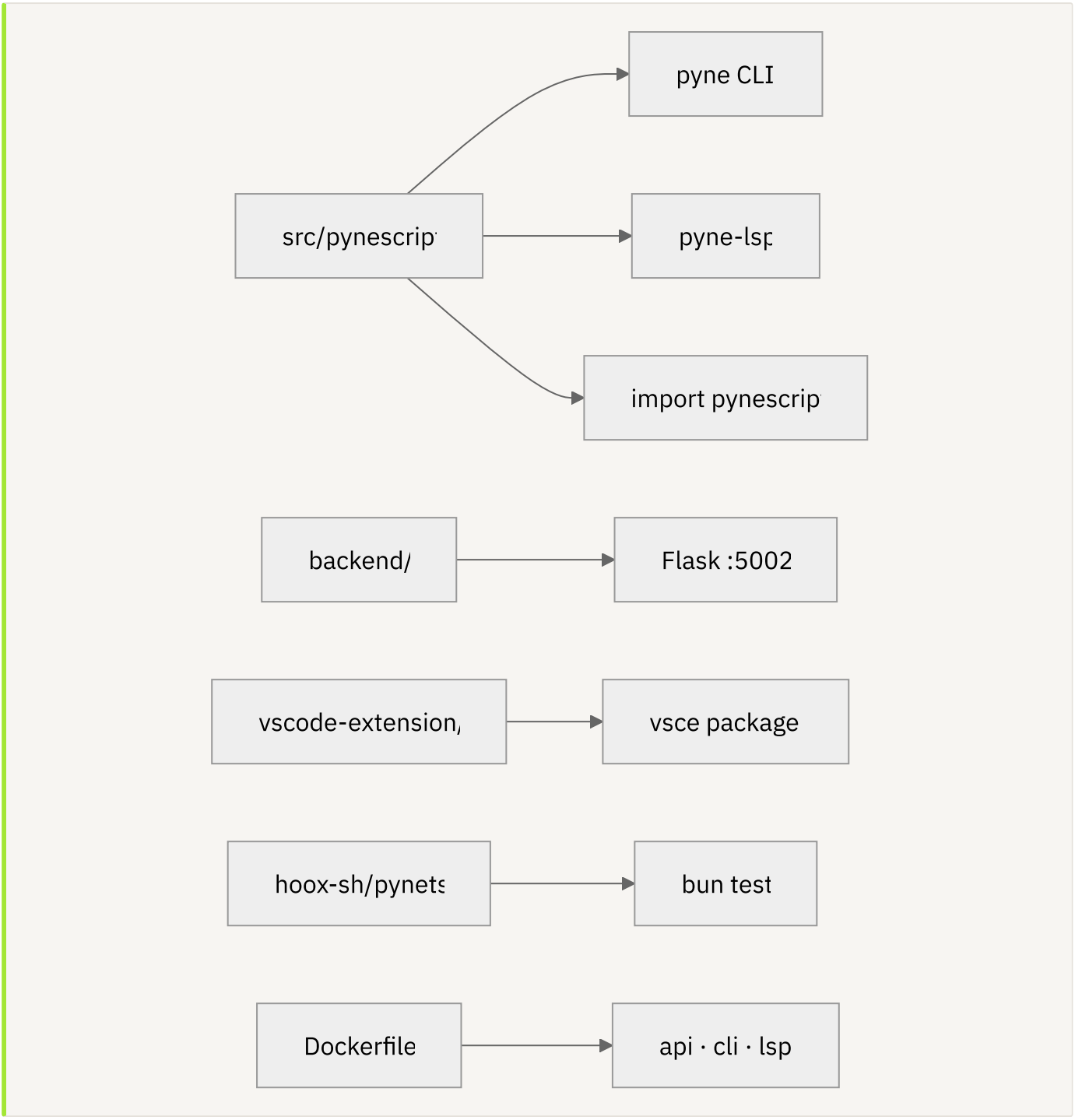
title: "Local development" description: "Install, test, lint, package, and run PYNE — Make targets after AXIS extraction."

LOCAL DEVELOPMENT

Abstract

A PYNE checkout is a **Python-first workspace**: src-layout package + Flask Pro API, optional VS Code extension (Node). PyneTS is the standalone hoox-sh/pynets / `@hoox-sh/pynets` package. The AXIS charting PWA is a **sister repo** (axis) — do not recreate `frontend/` here.

Conceptual model



Interface surface

Make targets (primary)

Target	Effect
`make install`	`pip install -e ".[lsp,pro]"`
`make test`	full `pytest tests/`
`make test-cli`	`tests/test_cli.py`
`make test-lsp`	langserver + lsp_features
`make test-backend`	`tests/test_backend.py`
`make lint` / `make fmt`	Ruff on `src/` , `tests/` , `backend/`
`make package`	sdist + wheel (`python -m build`)
`make build`	Nuitka **LSP** binary
`make build-cli`	Nuitka **CLI** binary
`make build-check`	import check for lsp+cli (~30s)
`make build-vscode`	compile + package VSIX
`make run`	Flask Pro API (`:5002` typical)
`make run-lsp`	`python -m pynescript.langserver`
`make docker-build`	bake **api**
`make docker-build-cli`	bake **cli**
`make docker-build-lsp`	bake **lsp** (`pynescript-lsp:latest` ; GHCR name `ghcr.io/hoox-sh/pyne/lsp`)
`make docker-build-all`	api + api-dev + lsp + cli
`make docker-buildx`	multi-platform release bake (`api` + `cli` + `lsp`)
`make docker-push-ghcr`	`gh workflow run ghcr.yml`
`make docker-cli ARGS="…"`	compose profile cli (ephemeral)
`make docker-up`	compose API on `:5002`
`make docker-up-full`	+ redis profile
`make docker-prod`	gunicorn overlay (`ADMIN_TOKEN` required)
`make docker-smoke`	curl health on `:5002`
`make docker-down` / `docker-logs`	stop / logs

Hatch

```
hatch run test:test
hatch run test:test-cov
hatch run lint:style
hatch run lint:typing
hatch run lint:gen-parser
```

Quick desk loop

```
git clone https://github.com/hoox-sh/pyne.git
cd pyne
python3 -m venv .venv && source .venv/bin/activate
pip install -e ".[lsp,compile,pro]"
pyne info
pyne check examples/rsi_strategy.pine
make test-cli
make run   # Pro API
```

From PyPI (no checkout)

```
pip install "hoox-pyne[lsp,compile]"
pyne --version   # 0.6.22
```

Internals

Path	Why it matters
`src/pynescript/`	Editable package root
`tests/conftest.py`	Optional `--example-scripts-dir` only; no third-party corpus shipped
`backend/requirements.txt`	Pro API deps (also covered by `[pro]` extra)
hoox-sh/pynets	Standalone TS library (`@hoox-sh/pynets`); not an in-tree submodule
`vscode-extension/`	PYNE VS Code extension
`Dockerfile`	targets `api` , `api-dev` , `lsp` , **`cli`**

Python

- **Requires-Python:** `>=3.10`
- **CI matrix:** 3.10–3.13
- **Hatch test matrix:** 3.10–3.12 (no 3.13 cell)
- **Recommended:** 3.12+ with venv

Optional extras

Extra	Adds
`lsp`	pygls / lsprotocol → `pyne-lsp` (alias `pynescript-lsp`)
`compile`	numpy / numba
`data` / `datafeed`	ccxt
`pro`	Flask + gunicorn + matplotlib + redis + compile

Invariants

1. **No `frontend/` in this repo** — AXIS is external.

2. `from __future__ import annotations` required on new Python modules.
3. **Grammar edits** only under `resource/`; regenerate via hatch, don't hand-edit generated parser.
4. **Corpus fixture** runs hundreds of cases — prefer focused unit tests for CI paths.
5. **CLI Docker** runs as uid 1000; bind-mounts must be readable by others.

Failure modes

Symptom	Fix
`No module named pynescript`	activate venv; `pip install -e .` or `hoox-pyne`
Pro API ImportError flask	`pip install "hoox-pyne[pro]"` or backend requirements
Nuitka missing	`pip install nuitka`; `make build-check` first
Docker CLI can't read file	chmod host path; don't use 0700-only dirs

See also

- Installation
- CI
- Docker
- Nuitka build

CI

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Continuous integration" description: "GitHub Actions for lint, Python 3.10–3.13 tests, package build, Docker api+cli smoke, and VS Code extension."

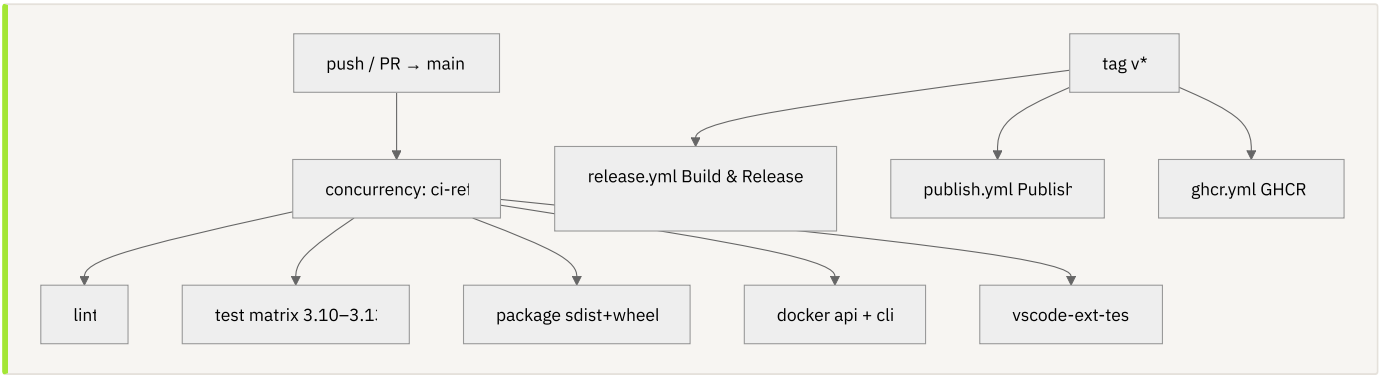
CONTINUOUS INTEGRATION

Abstract

CI is the **public invariant check** for the **pyne** repo after AXIS extraction. Workflows live under `.github/workflows/`. AXIS PWA/e2e CI lives in the sister axis repo — **not** here.

Workflow file	`name:`	Role
`ci.yml`	**CI**	Lint, test matrix, package, Docker api+cli, VSIX
`release.yml`	**Build & Release**	Tag `v*` + dispatch: wheels, CLI/LSP Nuitka binaries, CLI image tarball, VSIX, GitHub Release
`publish.yml`	**Publish**	Tag `v*` + dispatch: PyPI `hoox-pyne`
`ghcr.yml`	**GHCR**	Tag `v*` + dispatch: `ghcr.io/hoox-sh/pyne/{api,cli,lsp}` multi-arch
`docs-versions.yml`	**Docs versions**	Daily: refresh PyPI / npm / Open VSX / GitHub release stamps. Commits only on change. No docs pageload.

Conceptual model



Interface surface

Triggers

Workflow	On	Purpose
`CI`	push/PR to `main`	Required confidence
`Build & Release`	tags `v*` + dispatch	Binaries + VSIX + GitHub Release
`Publish`	tags `v*` + dispatch (`dry_run` default true)	PyPI `hoox-pyne`
`GHCR`	tags `v*` + dispatch	Public images `api` / `cli` / `lsp`

`ci.yml` jobs (current)

Job	Runtime	What it asserts
lint	Python 3.13	Ruff E/F/W gate on `src/` `tests/` (+ auth middleware); mypy non-blocking
test	Matrix 3.10–3.13	`.[lsp,pro,compile]` + backend deps; core (`test_linter` / `test_evaluator` / `test_cli`); runtime (parity, strategy, series, incremental TA, package Runtime); LSP (`test_langserver` + `test_lsp_features`); backend; Codecov on 3.13
package	3.13	`python -m build` + twine; wheel smoke (`pynescript --help`); artifact
docker	buildx	Build **api** + **cli** targets; API health + admin fail-closed; CLI `--help` / `info` / `check` (`--network=none`)
vscode-ext-test	Node 22	`npm ci` → compile → `vsce package` → VSIX artifact

Concurrency: `group: ci-${{ github.ref }}` , `cancel-in-progress: true` . Default permissions: `contents: read` .

Not in this repo's CI

- AXIS PWA / Playwright (axis repo — no `axis-nightly.yml` here)
- GHCR push (separate `ghcr.yml` , tag-triggered)
- Cloudflare Worker deploy
- Full `tests/` corpus parametrized suite (run locally: `make test`)

Internals

Path	Role
`.github/workflows/ci.yml`	PR/main fan-out (`name: CI`)
`.github/workflows/release.yml`	Multi-artifact tag release (`name: Build & Release`)
`.github/workflows/publish.yml`	PyPI `hoox-pyne` on `v*` tags (`name: Publish`)
`.github/workflows/ghcr.yml`	Multi-arch image push (`name: GHCR`)
`tests/test_cli.py`	CLI unit suite (CI-critical)

Secrets

`publish.yml` uploads `hoox-pyne` to PyPI on `v*` tags. LSP binary encrypt uses `CRYPTO_KEY` (see Metadata crypto). GitHub Releases use the default Actions token.

Invariants

1. **Fail-fast is off** on the Python matrix.
2. **Codecov only on 3.13**.
3. **CLI Docker image is non-root** — smoke mounts must be world-readable (`chmod` on mktemp).
4. **Corpus parametrized tests are intentionally skipped** in CI for time; local `make test` is broader.
5. **Package smoke** installs the wheel and runs `pynescript --help` .

Worked examples

```
# Lint as CI (correctness gate)
ruff check src/ tests/ backend/middleware/auth.py --select E,F,W --ignore E501,E741,F841,W293,E402

# Core unit slice
pip install -e ".[lsp,pro,compile]"
python -m pytest tests/test_linter.py tests/test_evaluator.py tests/test_cli.py -q

# Docker CLI smoke
docker buildx bake cli
docker run --rm --network=none pynescript-cli:latest info
```

Failure modes

Symptom	Cause	Fix
Docker CLI PermissionError on smoke.pine	mktemp 0700 + non-root user	CI chmods 755/644 (already fixed)
Wheel missing `pynescript`	wrong package name	install `hoox-pyne` / built wheel
Ruff F821 on main	incomplete imports	fix before push; gate is E,F,W

See also

- Release
- Contributing
- Docker

RELEASE

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Release" description: "Tag-driven CLI + LSP multi-platform Nuitka binaries, wheels, Docker CLI image, VSIX packaging, GitHub Releases, and Marketplace publish."

RELEASE

Abstract

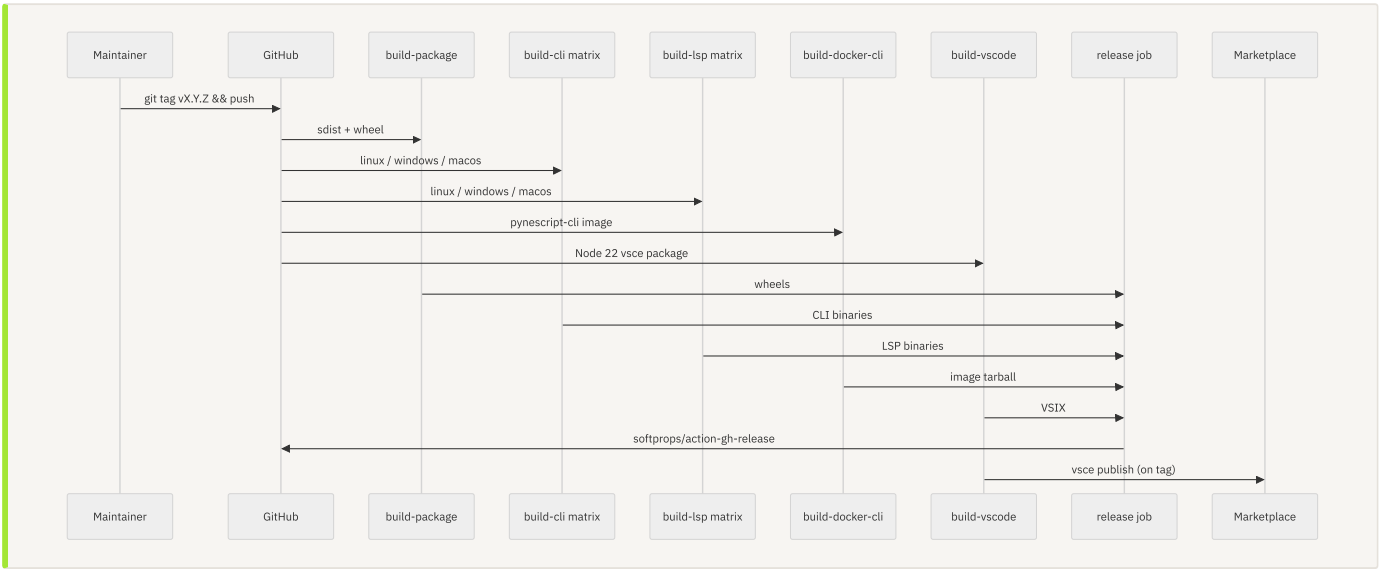
A PYNE release is a **multi-artifact event**, not a single wheel upload. The primary automation (`.github/workflows/release.yml`) builds:

- **PyPI sdist + wheel** (`hoox-pyne`) with CLI smoke
- **CLI** platform-native Nuitka binaries (`pynescript`)

- **LSP** platform-native Nuitka binaries (`pynescript-lsp`)
- **CLI Docker image** tarball (`pynescript-cli-image.tar.gz`)
- **VS Code** VSIX

Assets attach to a GitHub Release; PyPI upload is handled by **Publish** (`publish.yml`). Multi-arch container images are a **separate** workflow (**GHCR** / `ghcr.yml`) — not attached as tarballs except the CLI image save in this workflow. Marketplace / Open VSX publish is optional when those jobs are configured.

Conceptual model



Interface surface

Triggers

Event	Behavior
Push tag matching `v*`	Full build + create release + marketplace/Open VSX publish
`workflow_dispatch` with `version` input	Same jobs; **Create Release** also runs on dispatch (not tag-only)

Jobs

Job	Matrix / OS	Output
`build-package`	ubuntu-latest	`hoox-pyne-dist` (sdist + wheel)
`build-cli`	ubuntu-latest, windows-latest, macos-latest	`pynescript-cli-linux-x86_64` , `…-windows-x86_64.exe` , `…-macos-arm64`
`build-lsp`	ubuntu-latest, windows-latest, macos-latest	`pynescript-lsp-linux-x86_64` , `…-windows-x86_64.exe` , `…-macos-arm64`
`build-docker-cli`	ubuntu-latest	`pynescript-cli-docker` (gzipped `docker save`)
`build-vscode`	ubuntu-latest	`pynescript-vscode-extension` VSIX
`release`	needs package + vscode success; tag **or** dispatch	GitHub Release body + staged assets (`name: pyne vX.Y.Z`)
`publish-vscode`	needs VSIX; tag **or** dispatch	`vsce publish` / `ovsx publish` when configured

Environment

```
env:
  PYTHON_VERSION: "3.11"   # Nuitka 2.5–2.7 graph; comment in workflow
  NUITKA_JOBS: 4
```

Build steps rely on:

```
# LSP
pip install -e ".[lsp]" "nuitka>=2.5.1,<2.8" cryptography
python scripts/build/compile.py --target lsp --no-encrypt --check
python scripts/build/ci_build.py --target lsp --jobs 4
# CRYPTO_KEY from secrets.METADATA_KEY

# CLI
pip install -e . "nuitka>=2.5.1,<2.8"
python scripts/build/compile.py --target cli --check
python scripts/build/ci_build.py --target cli --jobs 4 --skip-metadata --skip-vsix
```

Local make-side packaging

```
make package        # sdist + wheel (python -m build)
make build          # Nuitka LSP
make build-cli      # Nuitka CLI
make build-vscode   # npm install && compile && vsce package
make docker-build-cli
```

Artifacts land under `dist/` , `dist/lsp/` , `dist/cli/` , `dist/vsix/` , and `vscode-extension/*.vsix` depending on script path.

Internals

Path	Role
`.github/workflows/release.yml`	Orchestration
`scripts/build/ci_build.py`	CI-oriented Nuitka + metadata encrypt
`scripts/build/compile.py`	Local/full compile options (`--onefile`, `--standalone`, `--check`)
`vscode-extension/package.json`	Extension version / engines
`src/pynescript/__about__.py`	Hatch dynamic version for Python package

Release asset contract (from release body)

Artifact	Install sketch
`hoox_pyne-*.whl` / `*.tar.gz`	`pip install ./hoox_pyne-*.whl`
`pynescript-cli-linux-x86_64`	`chmod +x` → `/usr/local/bin/pynescript`
`pynescript-cli-windows-x86_64.exe`	Place on PATH as `pynescript.exe`
`pynescript-cli-macos-arm64`	Place on PATH (Apple Silicon runners)
`pynescript-cli-image.tar.gz`	`gunzip -c … | docker load` → `pynescript-cli:latest`
`pynescript-lsp-linux-x86_64`	`chmod +x` → `/usr/local/bin/pynescript-lsp`
`pynescript-lsp-windows-x86_64.exe`	Place on PATH
`pynescript-lsp-macos-arm64`	Place on PATH (Apple Silicon runners)
`pyne-vscode-*.vsix` (`hoox-sh.pyne` 0.4.5)	`code --install-extension pyne-vscode-*.vsix` · site: hoox.sh/pyne-vscode.vsix (unversioned)

Invariants & edge cases

1. **`CRYPTO_KEY` must be stable across builds** if you care about byte-identical `builtin_metadata.json.enc`. Supply `secrets.METADATA_KEY`.
2. **Tag name is the version source** for the GitHub Release title (`v` prefix stripped).
3. **Marketplace / Open VSX publish is optional.** GitHub Release still attaches the VSIX when those jobs are unconfigured.
4. **Artifact retention is short** (5 days on build jobs) — releases must attach files immediately.
5. **macOS artifact is ARM64** from `macos-latest` — Intel Mac users may need separate builds if required.
6. **Draft vs prerelease:** workflow currently sets `draft: false`, `prerelease: false` for tag releases.

Worked examples

Cut a release

```
# ensure main is green
git checkout main && git pull
# bump versions in extension / __about__ as needed
git tag -a v0.6.22 -m "v0.6.22"
git push origin v0.6.22
# watch Actions → Build & Release, Publish, GHCR
```

Smoke-test a downloaded binary

```
chmod +x pynescript-cli-linux-x86_64
./pynescript-cli-linux-x86_64 --help
./pynescript-cli-linux-x86_64 check script.pine

chmod +x pynescript-lsp-linux-x86_64
./pynescript-lsp-linux-x86_64 --help
# or wire stdio into an editor client
```

Failure modes

Symptom	Cause	Fix
Binary not found after Nuitka	Path layout drift in `ci_build.py`	Inspect `dist/`; fix finder step
Decrypt errors in field	Key mismatch between encrypt and runtime	Align `METADATA_KEY` with embedded key strategy
Empty GitHub Release assets	Artifact download path mismatch	Match `files:` globs to download-artifact layout
Marketplace 401	Missing or expired marketplace credentials	Reconfigure the publish job
Windows `.exe` not marked executable	Finder uses `-type f -executable`	Fallback non-executable find branch in workflow

See also

- Nuitka build
- Metadata crypto
- CI
- VS Code extension

DOCKER

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

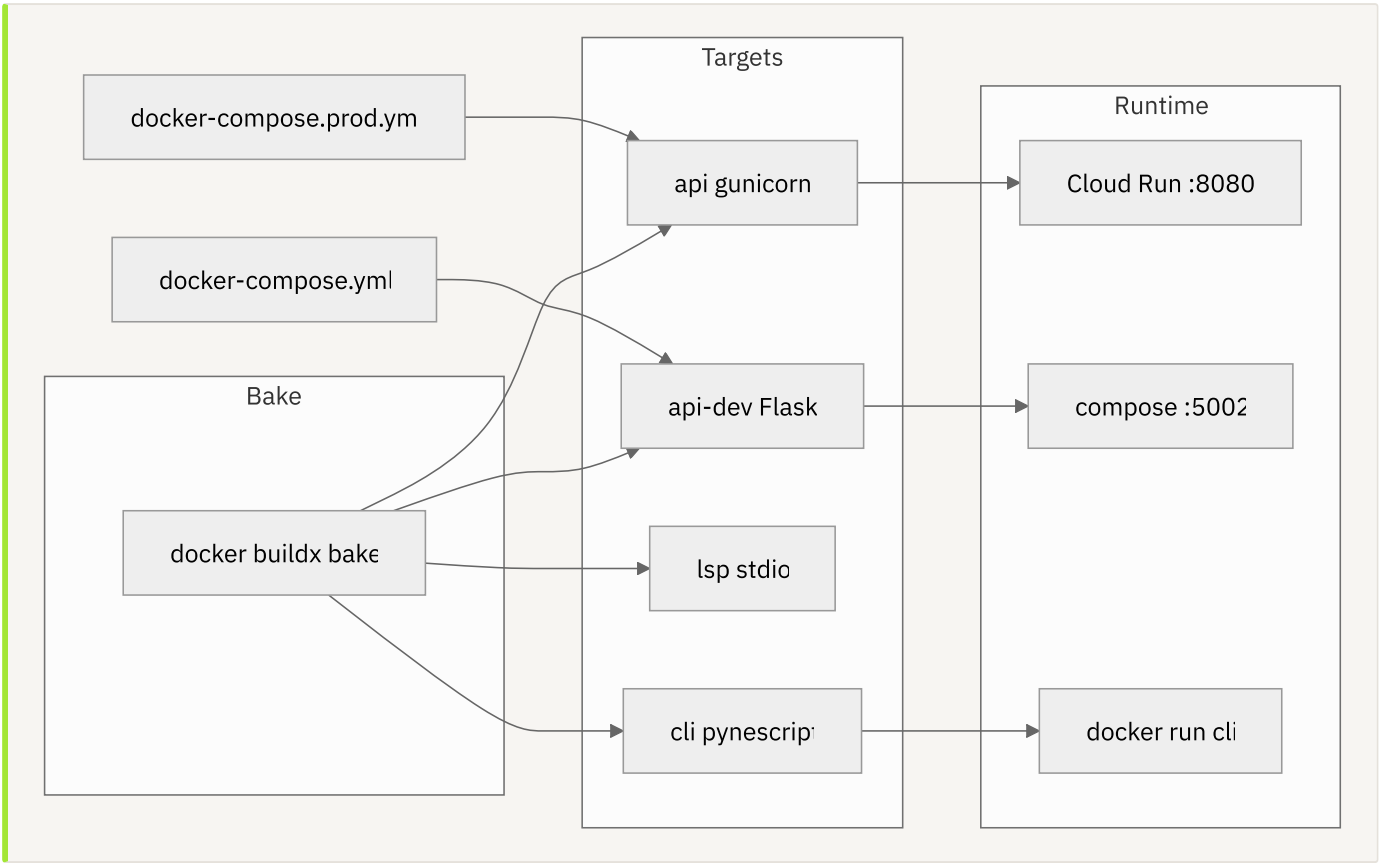
title: "Docker" description: "Buildx multi-target images and Compose profiles for the PYNE Pro API."

DOCKER

Abstract

Containers package the **Pro API** (Flask + evaluator), the **CLI** (`pynescript`), and the **LSP** for reproducible local runs and Cloud Run deploys. A single multi-stage `Dockerfile` exposes named targets; **Docker Buildx Bake** (`docker-bake.hcl`) drives local loads and multi-platform release builds. Compose wires volume mounts and optional Redis/LSP/CLI profiles for desk development, with a production overlay for gunicorn.

Conceptual model



Interface surface

Dockerfile targets

Target	Process model	Audience
`api`	`entrypoint-api.sh` → gunicorn (`0.0.0.0:$PORT`)	Production / Cloud Run / compose prod
`api-dev`	`python -m backend.app` with `HOST=0.0.0.0`	Local compose (source mounts)
`lsp`	`ENTRYPOINT pyne-lsp` (stdio)	Compose profile `lsp` ; GHCR `ghcr.io/hoox-sh/pyne/lsp`
`cli`	`entrypoint-cli.sh` → `pyne` (fallback `pynescript` ; ephemeral; `/work` CWD)	CI gates, one-shot parse/lint/compile/run

All targets: Python 3.12-slim, non-root `appuser` (uid 1000). API targets also set `API_KEY_STORE=/data/api_keys.json` , `MPLBACKEND=Agg` , healthcheck via `curl` . The **cli** image installs `.[compile,data]` only (no Flask/matplotlib) and is smaller/faster to build than `api` .

Buildx Bake (`docker-bake.hcl`)

```
# Local load (host platform)
docker buildx bake api                # production image → pynescript-api:latest
docker buildx bake cli                # CLI image → pynescript-cli:latest (ENTRYPOINT: pyne)
docker buildx bake lsp                # LSP image → pynescript-lsp:latest (ENTRYPOINT: pyne-lsp)
docker buildx bake                    # default group: api + api-dev
docker buildx bake all                # api + api-dev + lsp + cli

# Multi-platform (amd64 + arm64). Set REGISTRY to push:
REGISTRY=ghcr.io/hoox-sh/pyne TAG=0.6.22 docker buildx bake release
# release group = api-release + cli-release + lsp-release
# → ghcr.io/hoox-sh/pyne/{api,cli,lsp}:0.6.14
# Prefer: make docker-push-ghcr  (workflow GHCR on v* tags)
```

Make wrappers:

```
make docker-build       # bake api
make docker-build-cli   # bake cli
make docker-build-lsp   # bake lsp
make docker-build-all   # bake all targets
make docker-buildx      # bake release (multi-platform)
make docker-push-ghcr   # gh workflow run ghcr.yml
make docker-cli ARGS="check script.pine"   # compose profile cli
```

One-time multi-platform builder (if needed):

```
docker buildx create --use --name pynescript
```

Compose

```
cp .env.example .env   # optional overrides

make docker-up         # api-dev on host :5002
make docker-up-full    # + redis profile (api + redis)
make docker-prod       # requires ADMIN_TOKEN; gunicorn, no source mounts
make docker-smoke      # curl http://127.0.0.1:5002/
make docker-down

# stdio LSP (not a long-running compose service)
docker compose --profile lsp run --rm lsp

# CLI (ephemeral; mounts repo at /work)
docker compose --profile cli run --rm cli check script.pine
# ≡ make docker-cli ARGS="check script.pine"
```

Service	Profile	Notes
`api`	default	Port `${API_PORT:-5002}:8080`, ro mounts of `src/` + `backend/`, volume `api_data` → `/data`
`redis`	`redis`	redis:7-alpine, AOF, `${REDIS_PORT:-6379}`
`lsp`	`lsp`	stdio; use `docker compose run --rm lsp` (not detached `up`)
`cli`	`cli`	`entrypoint-cli.sh` → `pyne`; mounts `.:/work` + `src` + `cli_data`

Network name: `pynescript-dev`.

Production overlay (`docker-compose.prod.yml`): target `api`, **`volumes: !override`** so only `api_data:/data` remains (dev source binds are dropped), SQLite key store on `/data`, mem/cpu caps, requires `ADMIN_TOKEN`.

Environment variables

Variable	Default	Meaning
`PORT`	`8080`	Listen port inside the container
`HOST`	`0.0.0.0`	Bind address (dev Flask runner)
`API_PORT`	`5002`	Host port published to container `8080`
`FLASK_ENV`	`production` / `development`	App mode by target
`STORE_BACKEND`	`json` (prod overlay: `sqlite`)	`json` | `sqlite` | `redis`
`API_KEY_STORE`	`/data/api_keys.json`	JSON key store path (`STORE_BACKEND=json`)
`API_KEY_STORE_SQLITE`	`/data/api_keys.db`	SQLite path (`STORE_BACKEND=sqlite`)
`ALLOWED_ORIGINS`	compose default includes `https://hoox.sh`, `https://hoox.sh/axis`, and **local AXIS `:8081`**	CORS allow-list
`ADMIN_TOKEN`	unset	Required for `POST /auth/create_key` (`X-Admin-Token`) and prod compose
`PYNE_RUNNER`	`0`	Optional hosted runner (docs). `1` enables `/scripts` + `/cron/*`
`PYNE_RUNNER_SCHEDULER`	`0`	In-process bar-close poll. Prefer `GUNICORN_WORKERS=1` when on
`PYNE_RUNNER_DB`	`/data/runner.db`	SQLite on the `api_data` volume
`REDIS_URL`	empty in prod; compose dev default `redis://redis:6379/0`	Required when `STORE_BACKEND=redis`
`GUNICORN_WORKERS` / `GUNICORN_THREADS` / `GUNICORN_TIMEOUT`	`1` / `100` / **`120`** (entrypoint + prod compose; worker class hard-pinned `gthread`)	Prod entrypoint knobs
`GUNICORN_BIND`	`0.0.0.0:${PORT}`	Optional gunicorn bind override

Published image names

GitHub Container Registry (workflow **GHCR**, tags `v*`):

```
ghcr.io/hoox-sh/pyne/api:0.6.22
ghcr.io/hoox-sh/pyne/cli:0.6.22
ghcr.io/hoox-sh/pyne/lsp:0.6.22
```

Internals

Build stages

1. **base-os / base** — slim image, non-root `appuser`; API/LSP `base` also installs `curl` + freetype/png
2. **builder** — `build-essential`, pip cache mounts, install `backend/requirements.txt` then `pip install ".[lsp]"` into `/install`
3. **lsp-builder** — `pip install ".[lsp]"` only (no Flask / matplotlib)
4. **cli-builder** — `pip install ".[compile,data]"` only (no Flask stack)
5. **runtime / api / api-dev / lsp / cli** — final CMDs / ENTRYPOINTs and healthchecks (`lsp` has none — stdio)

`.dockerignore`

Keeps context small by excluding `tests/` (corpus), `docs/`, node modules, compiler `*.nbc` / `*.nbi` caches, venvs, and IDE cruft.

Compose healthchecks

API: `curl -fsS http://127.0.0.1:8080/`. Redis: `redis-cli ping`.

Invariants & edge cases

1. **Host port 5002 vs container 8080.** Local compose publishes API on **:5002**. Cloud Run uses **:8080**. Local AXIS PWA is **:8081**.
2. **Read-only mounts in compose** mean dependency changes need image rebuild; source edits are visible via `PYTHONPATH` precedence.
3. **Production target is immutable** — no live mounts; use the prod overlay or Cloud Run.
4. **Non-root `appuser`** cannot write arbitrary paths; keep `API_KEY_STORE` under `/data`.
5. **LSP is stdio** — not an HTTP service; use `docker compose run --rm lsp` rather than expecting a published port.
6. **CLI is ephemeral** — `entrypoint-cli.sh` execs `pyne` (fallback `pynescript`); pass subcommands after the image name. Workdir defaults to `/work` (compose mounts the repo there).
7. **Prod overlay is fail-closed** — `ADMIN_TOKEN` is required (`:?` interpolation); `volumes: !override` drops the dev source binds so only `api_data:/data` remains.

Worked examples

Production-like local API (bake)

```
docker buildx bake api
docker run --rm -p 8080:8080 \
  -e FLASK_ENV=production \
  -e ADMIN_TOKEN=change-me \
  -e ALLOWED_ORIGINS=https://hoox.sh,https://hoox.sh/axis \
  pynescript-api:latest
curl -s http://127.0.0.1:8080/ | jq .
```

Compose with Redis

```
docker compose --profile redis up --build
```

Optional LSP container

```
docker compose --profile lsp run --rm lsp
```

CLI image

```
docker buildx bake cli
docker run --rm pynescript-cli:latest --help
docker run --rm -v "$PWD:/work" -w /work pynescript-cli:latest check script.pine
docker run --rm -v "$PWD:/work" -w /work pynescript-cli:latest lint script.pine --json
# or via compose:
make docker-cli ARGS="run script.pine --bars 50"
```

Production overlay

```
docker compose -f docker-compose.yml -f docker-compose.prod.yml up --build -d
```

Failure modes

Symptom	Cause	Fix
Healthcheck fails	process not listening / curl missing	Use image targets from this Dockerfile; check `docker compose logs api`
Connection refused on host port	Flask bound to `127.0.0.1`	Dev target sets `HOST=0.0.0.0`; do not override to localhost
Permission denied writing keys	Running as `appuser` outside `/data`	Set `API_KEY_STORE=/data/...` and mount `api_data`
Import errors after mount	PYTHONPATH / stale site-packages	`PYTHONPATH=/app/src:/app`; rebuild after dependency changes
OOM under load	Small memory cap	Raise `API_MEM_LIMIT` / Cloud Run memory; reduce concurrency
CORS failures from PWA	Origin not allowed	Set `ALLOWED_ORIGINS`
Huge build context	Missing `.dockerignore`	Ensure `.dockerignore` is present (corpus/docs excluded)

See also

- Security
- Pro API lifecycle
- Local development

NUITKA BUILD

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Nuitka build (LSP + CLI)" description: "Compile pyne-lsp and pyne CLI with Nuitka — onefile vs standalone, CI build script, Anaconda quirks, and artifact layout."

NUITKA BUILD (LSP + CLI)

Abstract

Both the **Language Server** and the **Click CLI** can run as pure Python or as **Nuitka-compiled onefile binaries** for offline distribution. Compilation freezes the entry module and followed imports into artifacts under `dist/`.

Binary	Entry	Typical use
`pynescript-lsp` (PATH name; product CLI is `pyne-lsp`)	`langserver/__main__.py`	Editors / VSIX
`pynescript` (PATH name; product CLI is `pyne`)	`pynescript/__main__.py`	Shell / CI gates

Primary tools: `scripts/build/compile.py` (local) and `scripts/build/ci_build.py` (CI). Both accept `--target lsp|cli|all`.

Conceptual model



Interface surface

Commands

```
# Prerequisites
pip install nuitka cryptography

# Full LSP onefile build (default path via Make)
make build
# ≡ python scripts/build/compile.py --target lsp --jobs=4

# CLI onefile binary
make build-cli
# ≡ python scripts/build/compile.py --target cli --jobs=4

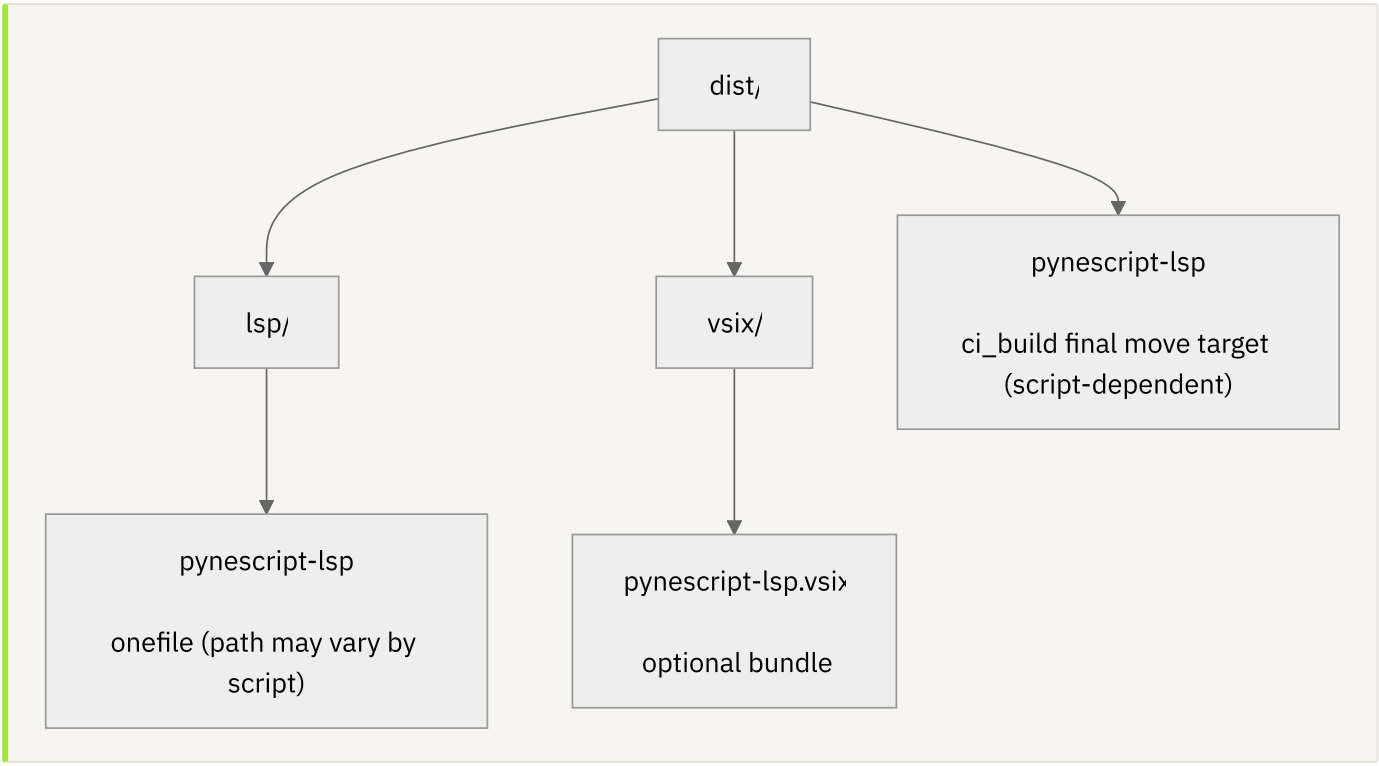
# Fast import check only (~30s) — both targets
make build-check
# ≡ python scripts/build/compile.py --target all --check

# Explicit modes
python scripts/build/compile.py --target cli --standalone
python scripts/build/compile.py --target all --jobs 8
python scripts/build/ci_build.py --target cli --jobs=4 --skip-metadata --skip-vsix
```

Build modes (approx. wall time)

Mode	Time	Notes
`--check`	~30s	Import resolution / no full compile
`--standalone`	5–15 min	Directory distribution, faster iteration
`--onefile`	10–30 min	Self-extracting single binary
CI 8 cores	5–15 min	High parallelism

Expected layout



Also see `scripts/build/README.md`.

Internals

Key Nuitka flags (`compile.py`)

- `--static-libpython=no` — default; critical for many Linux/Anaconda envs
- `--python-flag=no_site,no_docstrings`
- `--include-data-dir=…/providers=pynescript/langserver/providers`
- `--lto=auto`, `--jobs=N`
- `--onefile` or `--standalone`
- `--follow-imports` on local compile path

Entry module

Compilation targets `src/pynescript/langserver` (package entry), product name `pynescript-lsp`.

Metadata stage

Before compile, scripts may:

1. Run `scripts/generate_builtin_metadata.py` if plaintext metadata missing
2. Fernet-encrypt to `builtin_metadata.json.enc`
3. Write truncated SHA256 sidecar
4. Write `scripts/build/.metadata.key` (gitignored) or consume `CRYPTO_KEY` env in CI

Anaconda

If static libpython is unavailable:

```
conda install libpython-static
# or rely on --static-libpython=no (already the default in compile.py)
```

Invariants & edge cases

1. **C compiler required** (gcc/clang/MSVC). Containers need `build-essential` or equivalent if compiling inside Docker.
2. **Python 3.10+**; release workflow pins **3.11** with `nuitka>=2.5.1,<2.8`. Local sketches often use 3.12.
3. **Providers data dir must ship** — completion/hover collapse without metadata.
4. **Onefile startup cost** includes extract-to-temp; standalone is snappier for local soak tests.
5. **Do not commit** `scripts/build/.metadata.key`.

Worked examples

Local onefile with explicit jobs

```
python scripts/build/compile.py --jobs "$(nproc)"
find dist -name 'pynescript-lsp*' -type f
```

CI-shaped build

```
python scripts/build/ci_build.py --jobs=4
```

Verify without compile

```
python scripts/build/compile.py --check
```

Failure modes

Symptom	Cause	Fix
Link error on libpython	Anaconda static lib missing	Install static lib or keep `--static-libpython=no`
Binary missing builtins	Data dir not included	Confirm `--include-data-dir` providers path
Huge binary	Full stdlib pull-in	Review Nuitka plugins / unused deps
CI timeout	Low cores / onefile	Raise jobs; use larger runner; prefer standalone intermediate
Decrypt fail at runtime	Key not embedded / env unset	See Metadata crypto

See also

- Metadata crypto
- Release
- LSP builtin metadata

METADATA CRYPTO

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Metadata encryption" description: "Fernet-encrypted LSP builtin_metadata.json — key lifecycle, CI secrets, integrity hashes, and runtime decrypt."

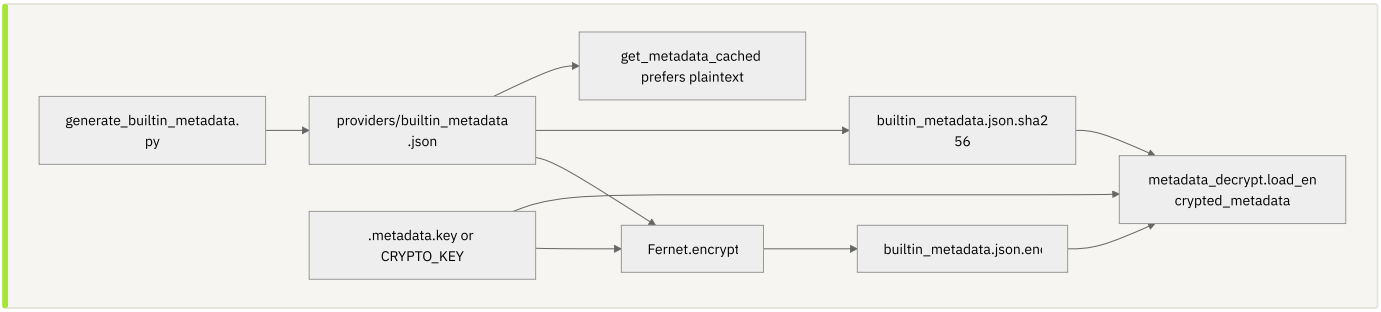
METADATA ENCRYPTION

Abstract

LSP completion, hover, and signature help depend on a large **builtin metadata** document derived from the live evaluator surface. In open development the JSON is plaintext and git-tracked. For **compiled Nuitka binaries**, the build pipeline can ship an encrypted blob so the binary does not leave a trivially scrapable metadata file on disk.

Mechanism: **Fernet** (symmetric, `cryptography` package) + optional SHA256 integrity sidecar.

Conceptual model



Interface surface

Files

Path	Tracked?	Role
`src/pynescript/langserver/providers/builtin_metadata.json`	yes	Plaintext for dev / hatch
`…/builtin_metadata.json.enc`	often yes	Encrypted payload for binaries
`…/builtin_metadata.json.sha256`	yes	Truncated SHA256 of plaintext
`scripts/build/.metadata.key`	**no** (gitignored)	Local Fernet key material
`scripts/generate_builtin_metadata.py`	yes	Regenerates JSON from code

Environment variables

Variable	Context	Meaning
`CRYPTO_KEY`	Build scripts / CI	Key bytes for encrypt stage
`PYNESCRIPT_METADATA_KEY`	Runtime decrypt fallback	Env-supplied key if file missing
`METADATA_KEY`	Build scripts	Alias for the same Fernet material

Code entrypoints

- Encrypt stage: `scripts/build/compile.py` (`encrypt_metadata` / `_resolve_fernet_key`), `scripts/build/ci_build.py` (`stage_metadata`)
- Decrypt: `src/pynescript/langserver/providers/metadata_decrypt.py`
- Loader preference: plaintext if present, else encrypted (`get_metadata` returns `{}` on miss; `get_metadata_cached` raises)

Internals

Generation (always code-derived)

```
python scripts/generate_builtin_metadata.py
```

Do **not** hand-edit the JSON for permanent feature work — re-run the generator after adding builtins.

Encrypt (local sketch)

```
from cryptography.fernet import Fernet
key = Fernet.generate_key()  # or load stable secret
fernet = Fernet(key)
encrypted = fernet.encrypt(plaintext_bytes)
```

Build scripts chmod the key file to `0o600` when writing `.metadata.key`.

Decrypt integrity check

After Fernet decrypt, if a `.sha256` sidecar exists, the loader compares `sha256(plaintext)[:16]` and raises on mismatch.

Dev vs binary

Mode	Behavior
Editable install / repo checkout	Prefer plaintext JSON
Nuitka onefile	Encrypted data dir + key resolution via MEIPASS / env
Missing both	`FileNotFoundError` directing to compile script

Invariants & edge cases

1. **Stable key ⇒ reproducible `.enc`.** Random key every CI run yields a different ciphertext even if plaintext is identical — not a functional bug, but pollutes diffs and caches.
2. **Plaintext remains in git** for open-source transparency of the *language surface*. Encryption protects the *bundled binary layout*, not the secret of the API catalog.
3. **Truncated hash is integrity, not secrecy.** It detects bitrot / wrong plaintext, not attackers with the key.
4. **Never commit `.metadata.key`.** Rotate if leaked; regenerate `.enc` with the new key for binary builds.
5. **After adding builtins:** regenerate metadata, re-encrypt with the **same** CI key, commit updated JSON (and `.enc` if tracked).

Worked examples

Regenerate + local encrypt

```
python scripts/generate_builtin_metadata.py
pip install cryptography
python scripts/build/compile.py --check   # stages metadata paths as configured
```

CI snippet

```
- name: Build LSP binary
  run: python scripts/build/ci_build.py --jobs 4
  env:
    CRYPTO_KEY: <stable-fernet-key>
```

Runtime with env key

```
pynescript-lsp
```

Failure modes

Symptom	Cause	Fix
`No metadata decryption key found`	Missing file + env	Supply `PYNESCRIPT_METADATA_KEY` or rebuild with key
`Metadata integrity check failed`	Stale hash or wrong ciphertext	Re-encrypt from current plaintext; commit both
Completion empty in binary only	Data dir not included in Nuitka	Fix `--include-data-dir`
Divergent `.enc` every CI	Unset `CRYPTO_KEY`	Configure `METADATA_KEY` secret

See also

- Nuitka build
- LSP builtin metadata
- Release
- Security

OBSERVABILITY

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

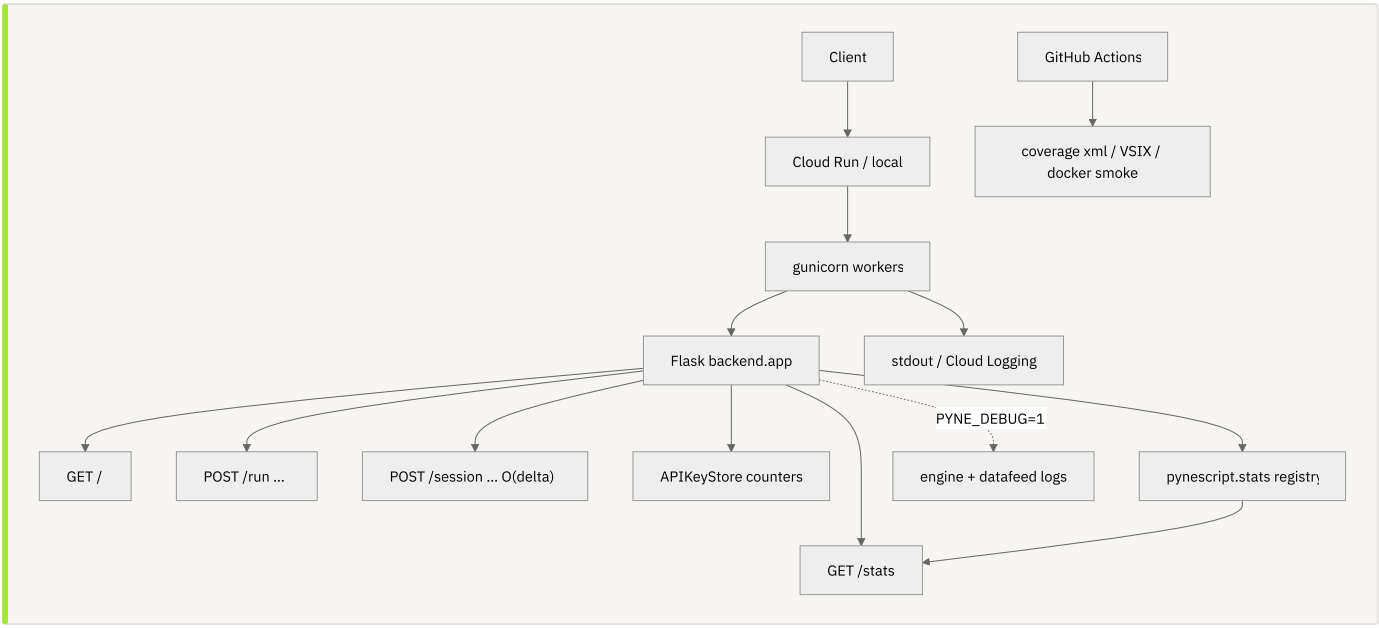
title: "Observability" description: "PYNE_DEBUG logging, GET /stats runtime counters, health endpoints, gunicorn process model, Cloud Logging, and operational signals."

OBSERVABILITY

Abstract

PYNE's production surface today optimizes for **simple, hostable signals** rather than a full OpenTelemetry mesh: HTTP health checks, **development debug logging** (`PYNE_DEBUG`), **process-wide runtime counters** (`GET /stats`), process managers (gunicorn), cloud platform logs, CI artifacts (coverage, Playwright reports), and application-level API key usage counters. Treat this page as the map of *what exists in-repo*, not a claim of enterprise APM parity.

Conceptual model



Interface surface

Health

`backend/app.py` exposes:

```
GET /
→ { "status": "healthy", "service": "pynescript-pro-api", … }
```

Used by:

- `Dockerfile` target `api` `HEALTHCHECK` (`curl -fsS http://127.0.0.1:8080/`)
- `docker-compose.yml` api healthcheck
- Load balancers / Cloud Run startup-ish probes (platform-dependent)

Runtime statistics

```
GET /stats        # public, no API key
→ { counters, latencies, gauges, compile_cache, sessions, debug }
```

Counters, latency percentiles (avg/min/max/p50/p95), and gauges for requests, runs, interpreter, compile, prewarm, live-session deltas, sessions, and datafeed. **Process-local** — resets on worker restart. Full reference: GET /stats.

```
curl -s http://127.0.0.1:5002/stats | python -m json.tool
```

Debug logging (`PYNE_DEBUG`)

A single development-only flag turns on detailed logging across the execution engine and the datafeed:

```
PYNE_DEBUG=1 python -m backend.app
PYNE_DEBUG=1 PYNE_DEBUG_SCOPE=pynescript.runtime,pynescript.compiler python -m backend.app
```

Variable	Default	Effect
`PYNE_DEBUG`	off	`DEBUG` on `pynescript` / `backend` loggers + all engine/datafeed instrumentation
`PYNE_DEBUG_SCOPE`	*(empty)*	Comma-separated logger allowlist; empty means no filter. Only consulted while `PYNE_DEBUG` is on.

What it traces:

Area	Logged
Interpreter	Bar-loop start/end with bars + ms + bars/ms; order-fill and runtime aborts with bar index/time; timeout breaks
Compile	Eligibility and the reason; host cache `HIT` / `MISS`; transpile (IR key, plots, object mode); JIT warm; nopython fallback
Prewarm	Builtins warm duration, per-script results, totals, failures
Delta calculation	`append_bars` delta size + start index; `update_last_bar` tick duration
Datafeed	CCXT Pro connect/subscribe/first updates/reconnects; REST `fetch_ohlcv` symbol normalization and bar counts; the `/datafeed/*` gateway; `WS /datafeed/watch` connect/close
Best-effort host plumbing	Intentionally swallowed errors surface as `ignored <what> (<Exc>) in <fn>()` — metrics, realtime scope snapshot/restore, plot-column extension, strategy-history commit, drawing/alert export
HTTP	Every request: method, path, status, duration; slow requests flagged

`PYNE_DEBUG=1 POST /run` on a fresh process, for example:

```
DEBUG pynescript.runtime.host:  auto: eligibility src=2de7f37e3736 eligible=True reason= (cached_fa
DEBUG pynescript.compiler.engine: prewarm: warming shared Numba builtins (first touch, JIT)
DEBUG pynescript.compiler.engine: prewarm: builtins warm in 7497.6ms
DEBUG pynescript.runtime.host:  compile: MISS src=2de7f37e3736 … in 8973.67ms host_cache_entries=1
DEBUG pynescript.runtime.host:  run finished mode=compile bars=40 total=8974.63ms parse=8973.67ms e
```

A warm second run shows the cache doing its job:

```
DEBUG pynescript.runtime.host:  compile: HIT src=2de7f37e3736 … in 0.00ms host_cache_entries=1
DEBUG pynescript.runtime.host:  run finished mode=compile bars=40 total=0.36ms parse=0.00ms eval=0.
```

And a live session shows the O(delta) property directly:

```
DEBUG pynescript.runtime.host:  append_bars run_id=250985e340… delta=2 start=20 new_total=22
DEBUG pynescript.runtime.host:  bar-loop start run_id=250985e340… start=20 end=22 bars=2 total_bars
DEBUG pynescript.runtime.host:  update_last_bar run_id=250985e340… idx=21 bar_time=… n_bars=22
```

`PYNE_DEBUG` is **development only** and verbose by design — it is meant to flood. Third-party loggers (`urllib3`, `numba`, `matplotlib`, `asyncio`, `werkzeug`) are held at `WARNING` while it is on. Keep it off in production: the numbers you want there come from `GET /stats`.

Implementation is three small pieces:

Path	Role
`src/pynescript/debug.py`	Env toggle, memoized on the raw env string so a live flip takes effect without a restart
`src/pynescript/stats.py`	Dependency-free metric registry
`backend/logging_setup.py`	Level policy, handler install, `debug_report()`

The toggle is read at run / batch / request boundaries, never inside a per-bar loop, and log payloads are only built when the flag is on.

Process model (production)

```
gunicorn --bind :8080 --workers 2 --threads 4 --timeout 120 backend.app:app
# docker/entrypoint-api.sh: GUNICORN_TIMEOUT default 120
# docker-compose.prod.yml: GUNICORN_TIMEOUT default 60
```

Knob	Effect on signals
workers	Process isolation; multiplies memory
threads	Concurrent requests within worker
timeout	Entrypoint default **120s**. A platform request limit tighter than that wins unless `GUNICORN_TIMEOUT` matches it.

Logs

Environment	Where logs go
Local `make run`	Process stdout/stderr
Docker	Container logs (`docker logs` , compose)

Application modules use standard library / Flask logging patterns; there is no mandatory structured-log schema enforced repo-wide. `` ` `` if a future branch introduces OpenTelemetry exporters.

CI / quality signals

Signal	Source
Ruff / mypy	`CI` lint job
Pytest results	runtime + LSP + backend steps in `CI`
Codecov (optional)	Python 3.13 matrix cell
VSIX	`CI` `vscode-ext-test` artifact
Docker smoke	`CI` api health + admin 403; CLI `--help` / `info` / `check`

Usage metering (app-layer)

`backend/middleware/auth.py` tracks per-key:

- `calls_used` / `calls_limit` / tier
- `last_used` timestamps

These are **business metrics** for rate limits, not Prometheus time series — export them if you need dashboards.

Internals

Path	Role
`backend/app.py`	Health route, `/stats`, request hooks, CORS, `MAX_CONTENT_LENGTH`
`backend/logging_setup.py`	`PYNE_DEBUG` level policy, handler install, `debug_report()`
`src/pynescript/debug.py`	Debug toggle + `PYNE_DEBUG_SCOPE` filter
`src/pynescript/stats.py`	Counter / latency / gauge registry behind `/stats`
`src/pynescript/runtime/host.py`	Interpreter, compile, delta debug + stats call sites
`src/pynescript/compiler/engine.py`	Compile / prewarm debug + stats call sites
`src/pynescript/util/datafeed.py` · `util/data.py`	Datafeed / provider debug + stats call sites
`backend/middleware/auth.py`	Key store, rate limit counters
`backend/middleware/sessions.py`	Live-session store + lifecycle counters
`backend/services/backtest.py`	Backtest metrics objects for API responses
`Dockerfile` (`api` target)	HEALTHCHECK + gunicorn entrypoint
`logs/` (repo)	Local combined/error log files if generated by tools — not the production contract

Invariants & edge cases

1. **Health ≠ readiness of evaluator warm caches.** A 200 on `/` does not mean the first `/run` will be fast (cold import / JIT / numba). Check `compile.host_prewarmed` / call `POST /compile/prewarm`.
2. **`/stats` is per-process, not fleet-wide.** Behind a load balancer each worker counts only its own slice. Aggregate per worker, or run single-worker (which live sessions require anyway).
3. **Timeouts double-bind.** A platform request limit shorter than gunicorn's default 120s wins unless `GUNICORN_TIMEOUT` is set to that limit.
4. **Scale-to-zero hides idle metrics.** No traffic ⇒ no samples; use synthetic uptime checks if SLOs matter.
5. **Coverage artifacts expire** (7–14 days retention on Actions uploads).
6. **Do not scrape secrets from logs.** API keys and admin tokens must never be logged at info level. `PYNE_DEBUG` logs *whether* credentials were supplied, never their values.
7. **Latency percentiles are recent-window.** `p50_ms` / `p95_ms` come from the last 512 samples; `max_ms` is all-time.

Worked examples

Local health + stats loop

```
make run
curl -s http://127.0.0.1:5002/    | python -m json.tool
curl -s http://127.0.0.1:5002/stats | python -m json.tool
```

Explain a slow run

```
PYNE_DEBUG=1 PYNE_DEBUG_SCOPE=pynescript.runtime,pynescript.compiler make run
# in another shell:
curl -s -X POST http://127.0.0.1:5002/run -H 'Content-Type: application/json' \
  -d '{"script":"//@version=6\nindicator(\"x\")\nplot(ta.sma(close,20))","data":[...]}'
```

Read the trace bottom-up: eligibility → cache HIT/MISS → JIT warm duration → `run finished total=…`. A first run paying ~9 s in `prewarm: builtins warm` and `compile: MISS` is expected; the second identical request should be a `compile: HIT` in ~0 ms.

Inspect live-session delta stats

```
curl -s http://127.0.0.1:5002/stats \
  | jq '{delta: .counters["delta.bars_appended"], appends: .counters["delta.append_calls"],
         max: .gauges["delta.bars_per_append_max"], loop: .latencies["interpret.bar_loop"]}'
```

A large `delta.bars_per_append_max` with a small `interpret.bar_loop.p95_ms` means the delta path is behaving as intended.

Docker health

```
docker inspect --format='{{json .State.Health}}' CONTAINER
```

Failure modes

Symptom	Cause	Fix
Health green, `/run` 500	Exception in evaluator path	Inspect request logs; reproduce with payload
First `/run` takes seconds	Cold Numba JIT	`POST /compile/prewarm` at deploy; confirm on `/health` `compile.host_prewarmed`
`/stats` counters reset	Worker restarted	Expected — in-process by design
`/stats` counts look low	Multiple workers, per-process counters	Aggregate per worker or set `GUNICORN_WORKERS=1`
No engine debug output	`PYNE_DEBUG` not set in the worker process	Set it in the environment, not only in your shell — gunicorn inherits from its parent
Too much debug output	No scope filter	Set `PYNE_DEBUG_SCOPE` to the modules you are debugging
Worker kills	Soft timeout	Optimize script / raise timeout consistently
Missing CI artifact	Path wrong / tests passed	`if-no-files-found: ignore` hides absence
Rate limit false positives	Shared key / limit too low	Adjust tier limits; multi-key

See also

- Docker
- Security
- API lifecycle

SECURITY

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

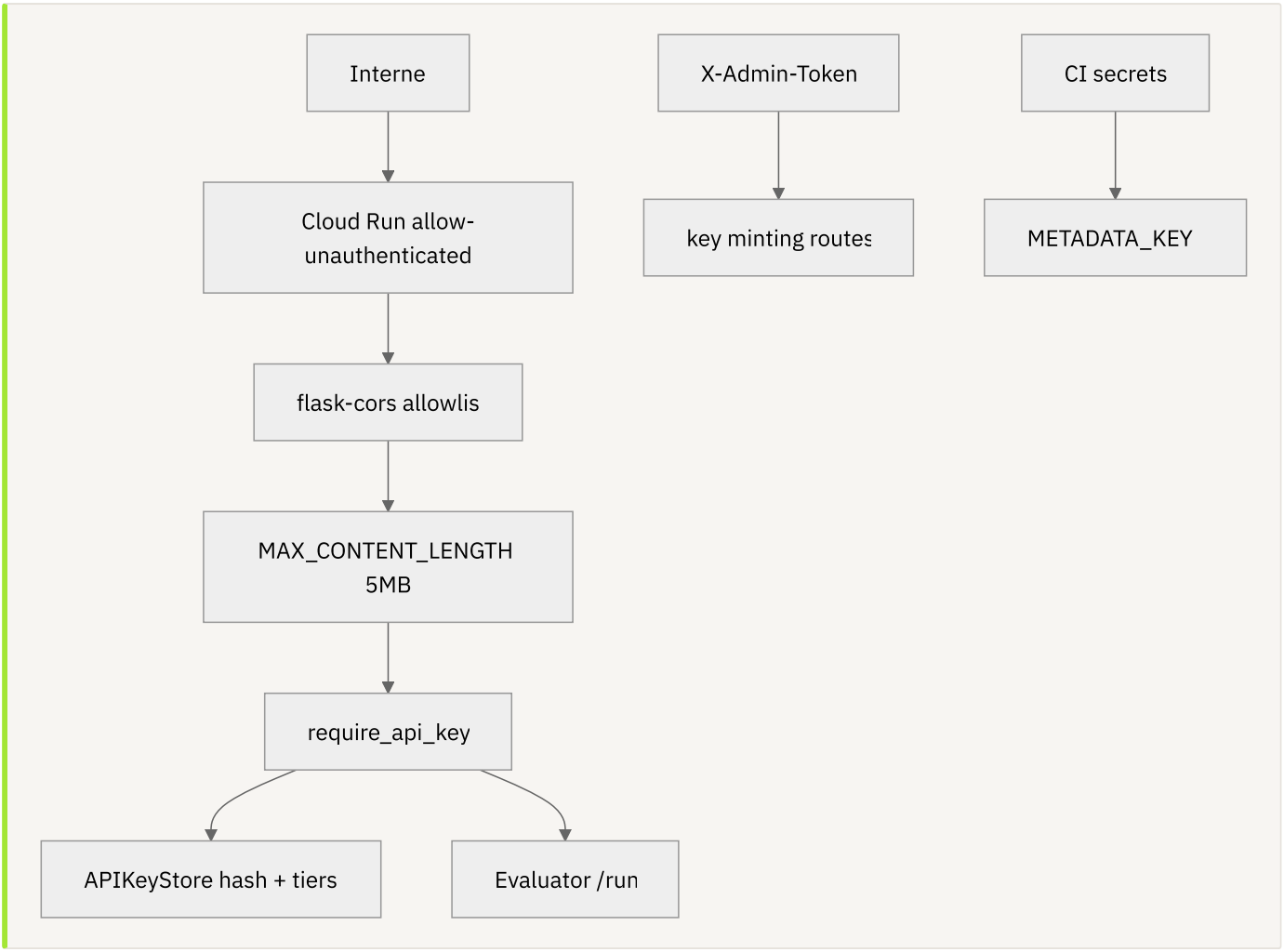
title: "Security" description: "API keys, CORS, body limits, secrets hygiene, Fernet metadata keys, and threat-aware defaults for PYNE ops."

SECURITY

Abstract

PYNE security is layered: **transport/hosting** (Cloud Run, containers), **application** (API keys, admin tokens, CORS, body size caps), **build secrets** (Fernet metadata keys), and **supply chain** (CI dependency installs, extension packaging). This page documents controls that exist in code and workflows — not a full threat model certification.

Conceptual model



Interface surface

HTTP application controls (`backend/app.py`)

Control	Default / mechanism
Body size	`MAX_CONTENT_LENGTH = 5 * 1024 * 1024` (5 MiB) — DoS/OOM mitigation
CORS	`ALLOWED_ORIGINS` env (comma-separated); default includes product hosts + localhost regex
Methods	GET, POST only via flask-cors config
Headers	`Content-Type` , `Authorization` , `X-Admin-Token`
Credentials	`supports_credentials=False`

API keys (`backend/middleware/auth.py`)

- Keys stored as **hashes**, not raw secrets (store path via `API_KEY_STORE`)
- Tiers: `free` , `hobby` (5k), `pro` (50k), `team` (200k), `enterprise` (∞)
- Rate limit by monthly call counters (`calls_used` / `calls_limit`)
- Decorators: `require_api_key` , `require_admin_token`

Build / release secrets

Secret	Purpose
`METADATA_KEY` / `_METADATA_KEY`	Stable Fernet key for LSP metadata encrypt
`GITHUB_TOKEN`	Release asset upload
`PYNESCRIPT_METADATA_KEY`	Runtime decrypt env fallback

Container hardening (`Dockerfile` target `api`)

- Multi-stage build (no compiler toolchain in final image)
- Non-root `appuser` (uid 1000)
- Production env flags + gunicorn entrypoint
- Healthcheck without privileged ops
- Intended mutable path `/data` (key store); prod compose uses SQLite there for multi-worker

Admin minting

- `POST /auth/create_key` requires `ADMIN_TOKEN` env **and** matching `X-Admin-Token` header (constant-time compare)
- Unset `ADMIN_TOKEN` → 403 (fail closed, not open access)

Key store backends (`STORE_BACKEND`)

Value	Use
`json` (default)	Single-process / tests; file may hold raw keys
`sqlite`	Multi-worker single host (shared volume); **hash-only**
`redis`	Multi-replica; needs `REDIS_URL` ; **hash-only**

Internals

Path	Role
`backend/app.py`	CORS, body limit, blueprint registration
`backend/middleware/auth.py`	Key store selection, tiers, decorators, admin gate
`backend/middleware/key_store_redis.py` / `key_store_sqlite.py`	Hash-only shared stores
`backend/middleware/schemas.py`	Request validation schemas
`src/pynescript/langserver/providers/metadata_decrypt.py`	Encrypted metadata load
`.github/workflows/*.yml`	Secret consumption (`METADATA_KEY`)
AXIS PWA security tests	Sister repo hoox-sh/axis — not in this tree

Audit notes encoded in source

Comments in `app.py` reference a **2026-07-05 audit** (findings S1 body size, S7/S8 CORS). Prefer reading the code over assuming defaults never change.

Invariants & edge cases

1. **Cloud Run `--allow-unauthenticated` does not mean open data.** It means IAM does not gate invoke; the Flask app must.
2. **Free tier with `calls_limit=0` is special-cased** as unlimited remaining in some helpers — verify tier semantics before relying on remaining-call counts.
3. **Localhost CORS regex** permits any port on localhost/127.0.0.1 — fine for desk, wrong for prod `ALLOWED_ORIGINS`.
4. **Fernet protects binary metadata packaging**, not user script confidentiality.
5. **Admin token routes are high privilege** — rotate and store separately from user API keys.
6. **Never log raw API keys.** Hash lookups only.

Worked examples

Harden CORS for a single origin

```
python -m backend.app
```

Point key store at a volume

```
# ensure process user can write /data
```

Rotate metadata key (ops procedure sketch)

1. Generate new Fernet key; store as secret `METADATA_KEY`
2. Rebuild LSP with `CRYPTO_KEY` set
3. Ship new binary; retire old key after client upgrade window
4. Do not leave dual-key decrypt unless you implement it

Failure modes

Symptom	Cause	Fix
Browser CORS error	Origin not listed	Set `ALLOWED_ORIGINS`
413 Payload Too Large	Body > 5MB	Reduce OHLCV batch; raise limit knowingly
401/403 on `/run`	Missing/invalid key	Check `Authorization` header scheme
Rate limited	Tier exhausted	Upgrade tier / wait reset
Secret in fork PR logs	Misconfigured workflow echo	Never `echo` secrets; use masked env
Root-owned files in volume	Old image ran as root	Align UID with `appuser`

See also

- Auth and keys (API)

- Metadata crypto
- Docker